HIGH-PERFORMANCE COMPUTING DOCUMENTATION

Astra

GPU & CPU Scheduling User Guide

A practical guide to running Python, machine-learning, and research workloads on Astra.

6 × A100
NVIDIA GPU devices

40 GB
VRAM per GPU

Up to 32
CPU cores per workload

Instructions prepared by **Deepak Naik M V**

am.en.r4cse23001@am.students.amrita.edu
deepaknaikmv01@gmail.com

Astra is accessible through designated systems in N111-A (HPC Lab).

GUIDE NAVIGATION

Table of Contents

1 About Astra

2 Getting Access

3 Prerequisites

4 Understanding Directories

5 Recommended Project Layout

6 Uploading Files and Data

7 Starting a Jupyter Session

8 Accessing the Terminal

9 Creating a Virtual Environment

10 Installing Dependencies

11 PyTorch and CUDA

12 Verifying GPU Availability

13 Preparing the Main Program

14 Creating the Scheduler Script

15 Scheduling a GPU Job

16 Selecting Queue and Resources

17 Monitoring Jobs and Outputs

18 Troubleshooting

19 Best Practices

20 Quick Reference

1. About Astra

Astra is Amrita's shared high-performance computing platform for GPU- and CPU-intensive workloads. It supports research, experimentation, model training, data processing, and other computationally demanding work.

A scheduling system allocates resources among users. Every job should request the CPU, GPU, memory, and execution resources it genuinely needs. Accurate requests improve scheduling efficiency and prevent shared capacity from being occupied unnecessarily.

6
NVIDIA A100 GPUs
Six GPU devices/cards are available across the platform.

40 GB
VRAM per GPU
Each A100 device provides 40 GB of GPU memory.

32
CPU cores
Up to 32 cores, subject to the request and resource availability.

SCOPE
This manual explains the documented Python and GPU workflow. The source material does not specify a separate CPU queue or CPU-only submission profile, so those settings are not invented here.

2. Getting Access to Astra

Open the Astra portal at https://astra.amrita.ac.in/pbsworks/login. If the login page does not load correctly in a normal browser session, try an Incognito or Private Browsing window.

Credentials

- **Faculty members:** request Astra credentials from HPC Support at hpc@am.amrita.edu.
- **Students and research scholars:** obtain access details through the relevant faculty supervisor or research guide.

ACCOUNT SECURITY
Do not share Astra credentials beyond authorized users. Follow institutional policies for shared computing resources.

3. Prerequisites

Most Python jobs need two core files. Additional datasets, pretrained models, checkpoints, configuration files, or other resources may also be required by the program.

01

`requirements.txt`

Lists the Python libraries and dependencies required by the project.

02

Main Python script

The entry point containing the program or experiment, such as `main.py` or `test.py`.

+

Project resources

Datasets, models, checkpoints, and related files used by the program.

> **KEEP PYTORCH SEPARATE**
>
> Do not include `torch`, `torchvision`, or `torchaudio` in `requirements.txt`. Install the tested CUDA-compatible builds separately, as described in Section 11.

4. Understanding the Astra Directory Structure

After login, Astra may initially display a path similar to `hpccluster/PBSHOME/stage/<username>`. Selecting the correct location is essential for reliable job execution and file management.

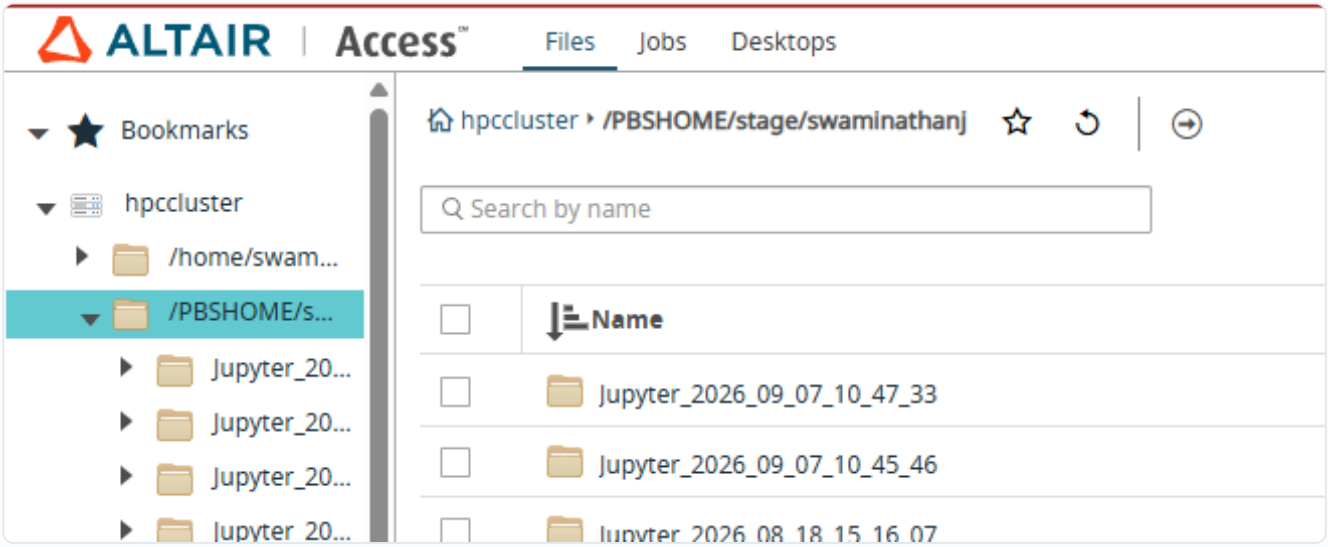


Figure 1. A typical `/PBSHOME/stage/<username>` view. The username visible in screenshots is an example account.

1 Navigate to the base directory

Click **hpccluster** in the breadcrumb path. At this level, two directories may appear to be associated with the same username.

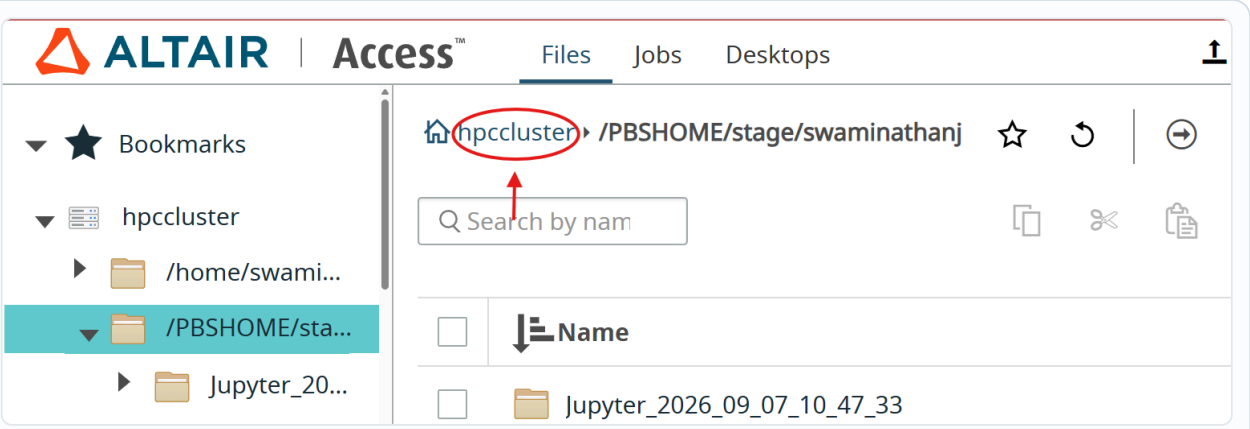


Figure 2. Use the **hpccluster** breadcrumb to return to the base directory.

2 Distinguish the two username directories

Open each directory and inspect its path. One is used mainly for PBS job execution and generated logs; the other is the user's main working directory.

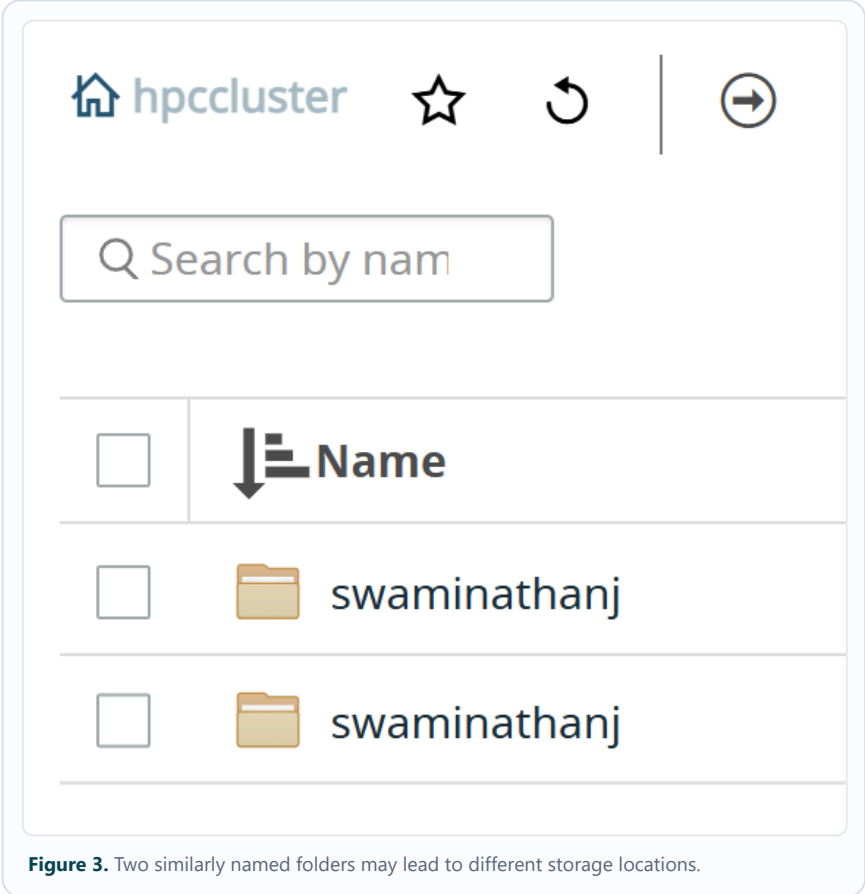


Figure 3. Two similarly named folders may lead to different storage locations.

Job staging and logs

`/PBSHOME/stage/<username>/Jupyter_<timestamp>`

Contains scheduler-generated job directories such as `Jupyter_YYYY_MM_DD_HH_MM_SS`. Do not use it as the primary project workspace.

Main working directory

`/home/<username>`

Store project folders, scripts, dependency files, datasets, models, and checkpoints here.

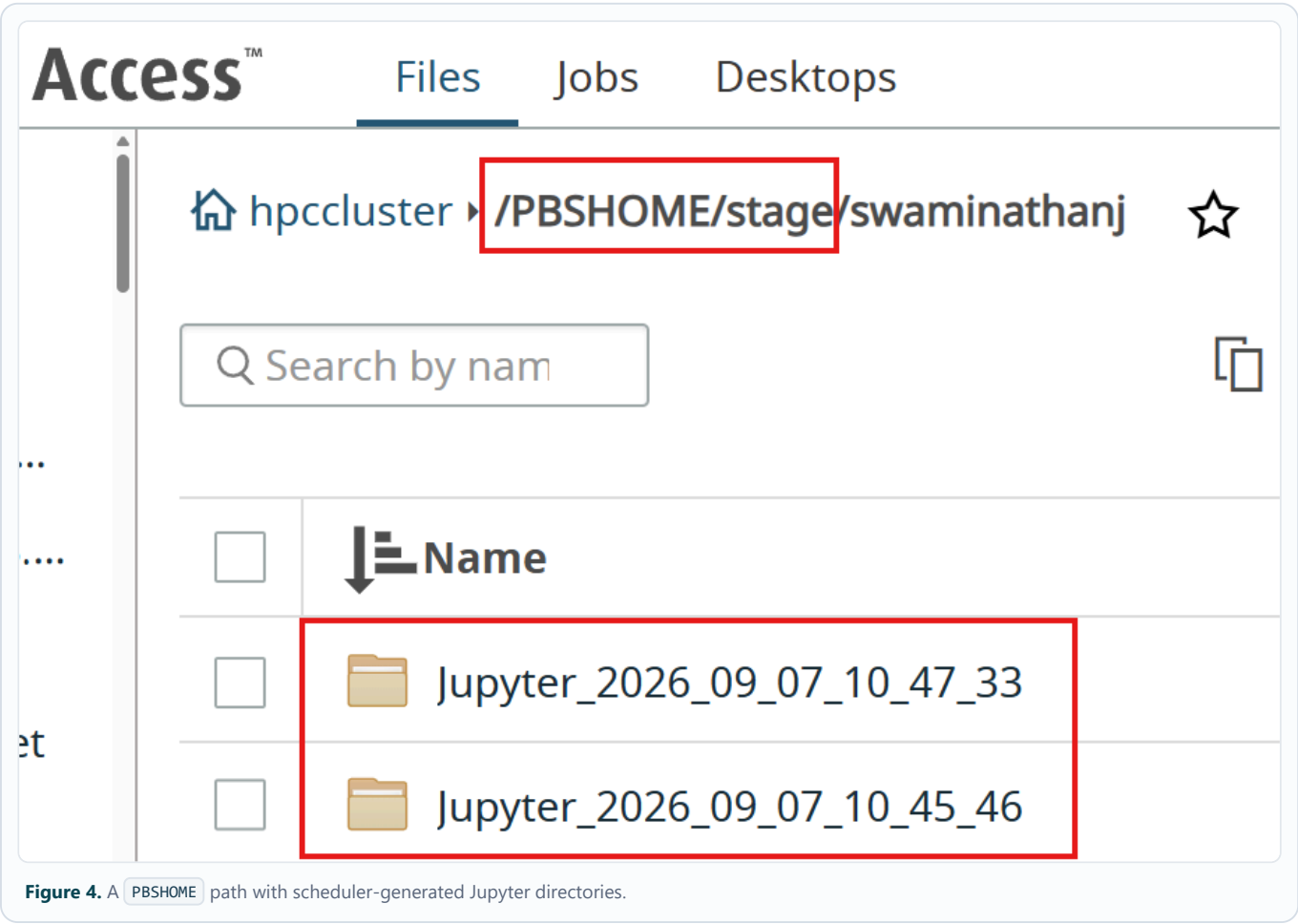


Figure 4. A `PBSHOME` path with scheduler-generated Jupyter directories.

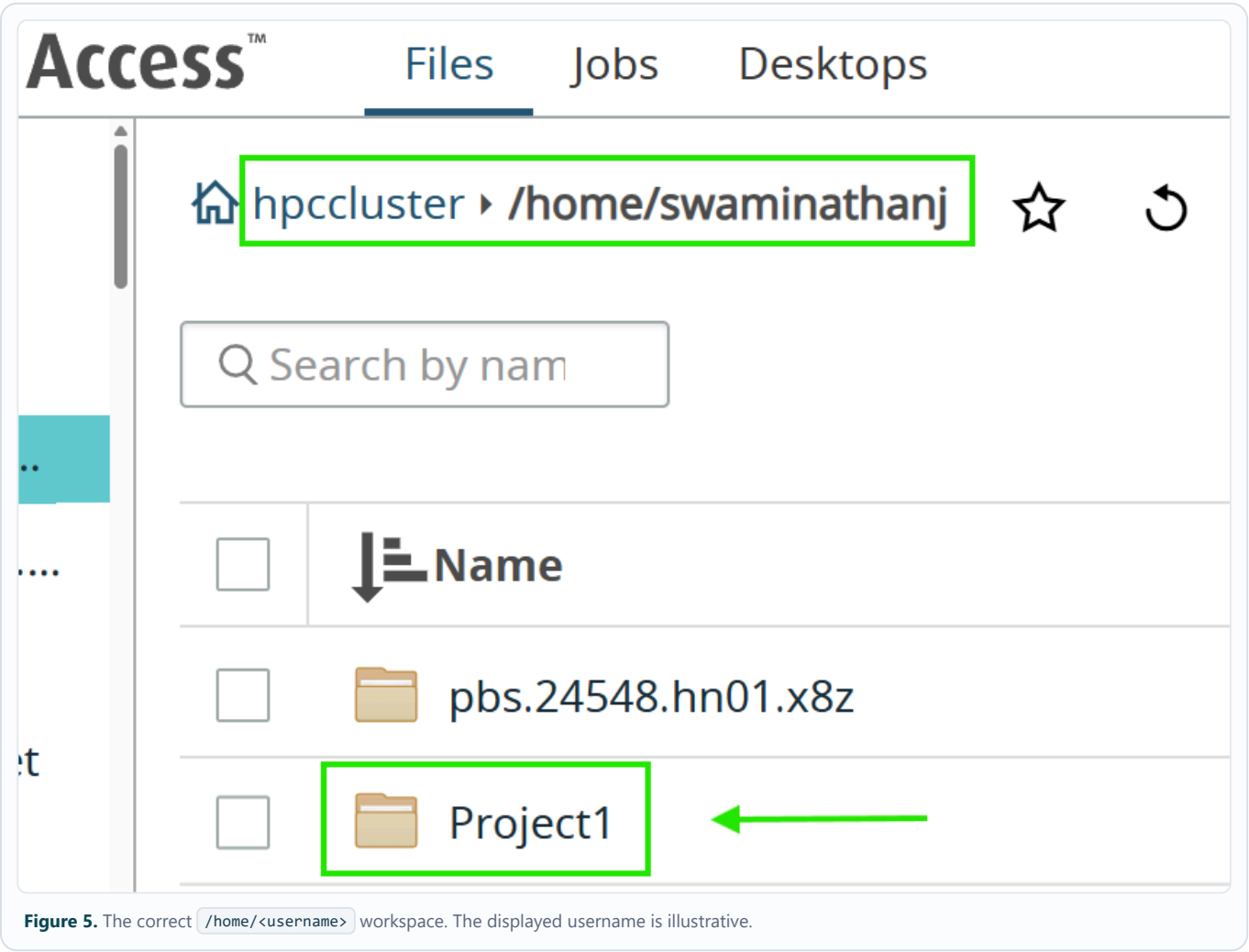


Figure 5. The correct `/home/<username>` workspace. The displayed username is illustrative.

> **IMPORTANT**
> Always create the project under `/home/<username>`, not under `/PBSHOME/stage/<username>`.

Create the example project

Inside `/home/<username>`, open the **New** menu at the top right, select **New Folder**, and create `Project1`. The resulting path is `/home/<username>/Project1`.

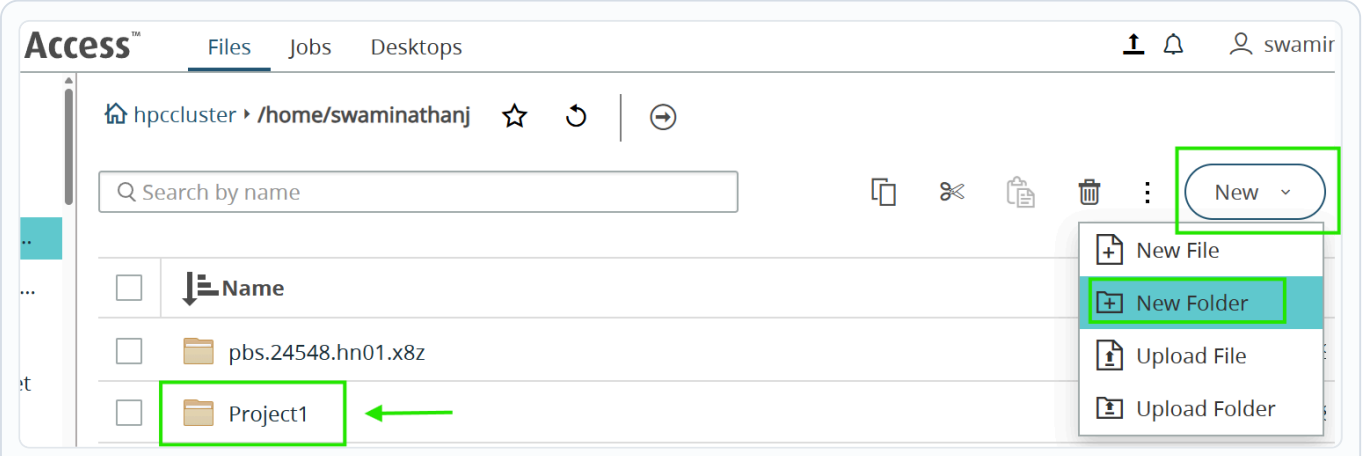


Figure 6. Create `Project1` from the **New** menu in the main home directory.

5. Recommended Project Directory Structure

The exact structure is optional, but separating code, models, data, and scheduler scripts makes experiments easier to manage and keeps generated job folders away from source files.

DIRECTORY TREE

```
/home/<username>/Project1/
├── env/
│   └── bin/
│       └── python
├── requirements.txt
├── content/
│   ├── files/
│   │   ├── main.py
│   │   └── test.py
│   ├── models/
│   │   ├── base_model/
│   │   └── checkpoints/
│   └── dataset/
│       └── dataset_files/
└── run_on_astra/
    ├── run_test.py
    ├── schedule_job2.py
    └── <automatically generated job folders>
```

- `content/files` — main Python scripts and other source files.
- `content/models` — base or pretrained models, checkpoints, fine-tuned models, and saved weights.
- `content/dataset` — datasets used by experiments.
- `run_on_astra` — scheduler/wrapper scripts and the job directories Astra may generate beside them.

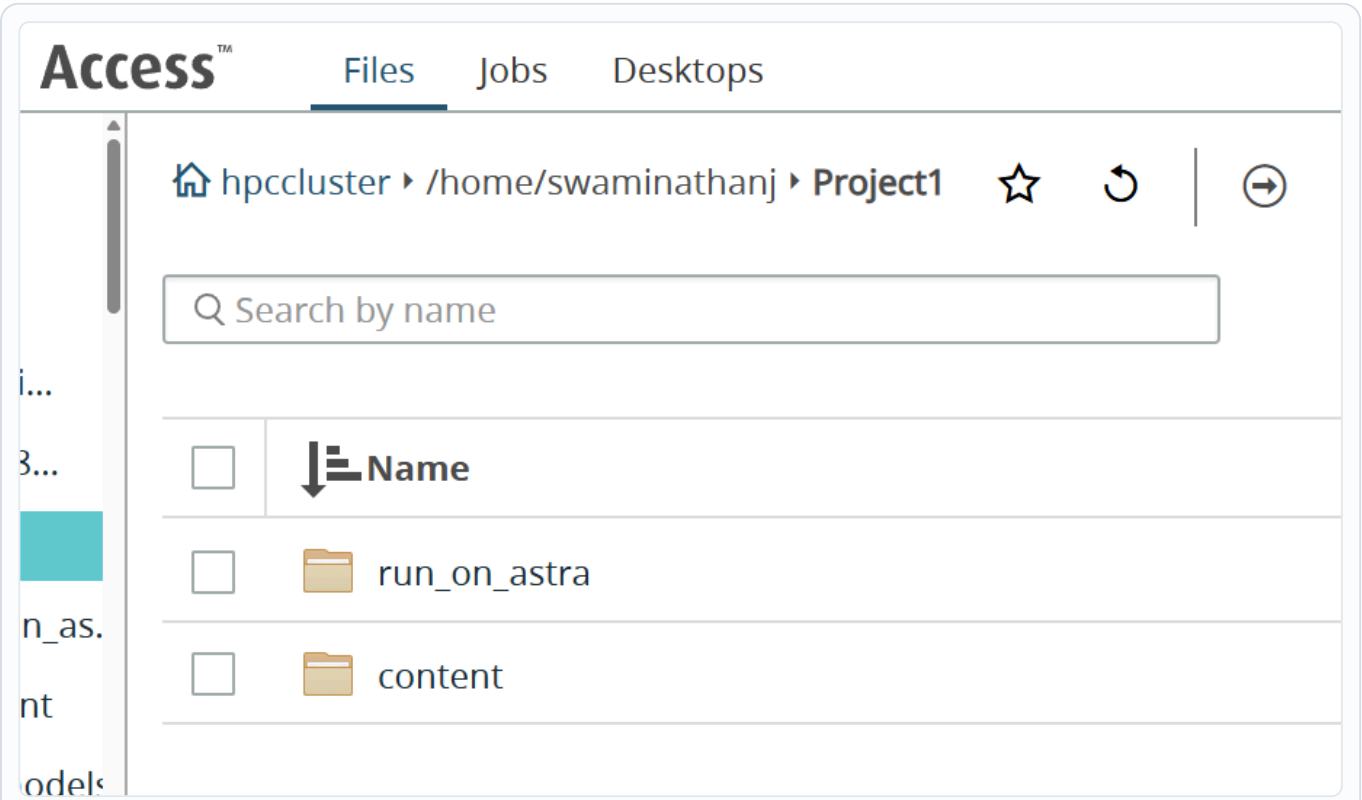


Figure 7. Top-level separation of scheduler files and project content.

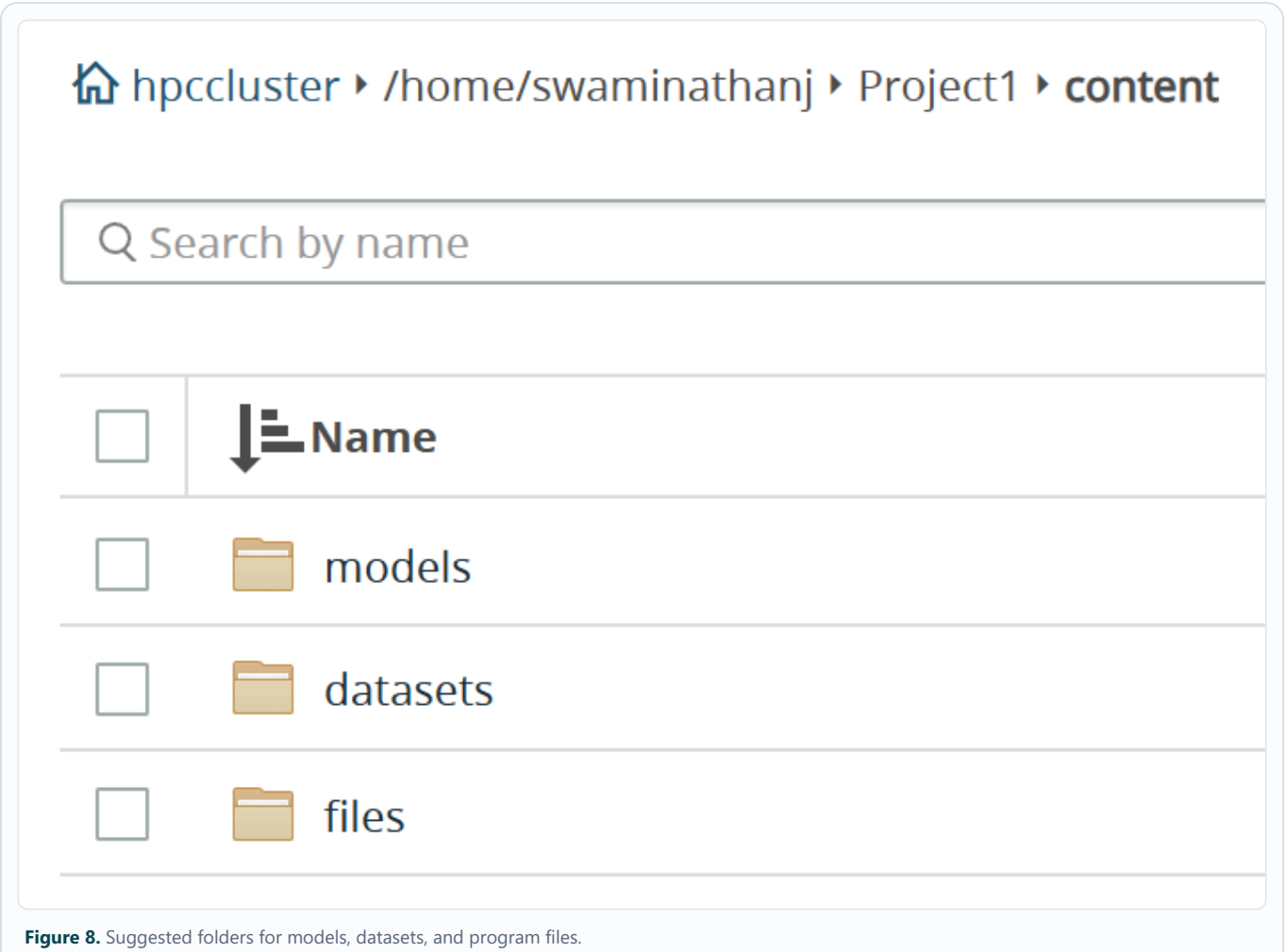


Figure 8. Suggested folders for models, datasets, and program files.

WHY SEPARATE SCHEDULER SCRIPTS?

Astra may create a job-related directory in the location from which a scheduler script is executed. Keeping wrappers in `run_on_astra` isolates that clutter from code, datasets, and model files.

6. Uploading Project Files, Datasets, and Models

Files smaller than 5 GB

Upload files directly from the local system through Astra's upload interface.

Files larger than 5 GB

Large files must be transferred through ICTS:

1. Copy the dataset, model, checkpoint, or other files to an external hard drive.
2. Through a faculty member, email ICTS System Care at syscare@am.amrita.edu.
3. State the exact source folder/file path on the drive and the absolute destination path on Astra.
4. Submit the external hard drive to ICTS.
5. Wait for the completion email, then collect the drive from ICTS.

BEFORE SUBMITTING A TRANSFER

Clearly specify both source and destination paths. Organize large resources first so ICTS can transfer the correct files without ambiguity.

7. Starting a Jupyter Session

A terminal is made available through a scheduled Jupyter session. Before starting, upload `requirements.txt` to the intended project directory—for example, `/home/<username>/Project1`.

1 Open the Jupyter job panel

From the main Astra interface, select **Jobs** → **Jupyter**.

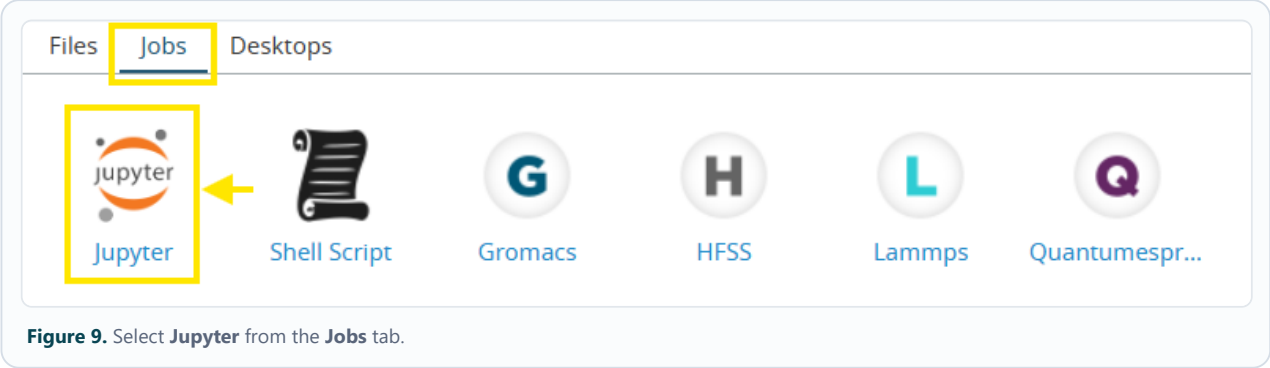


Figure 9. Select **Jupyter** from the **Jobs** tab.

2 Submit the Jupyter job

For a standard setup session, retain the documented default values and click **Submit**.

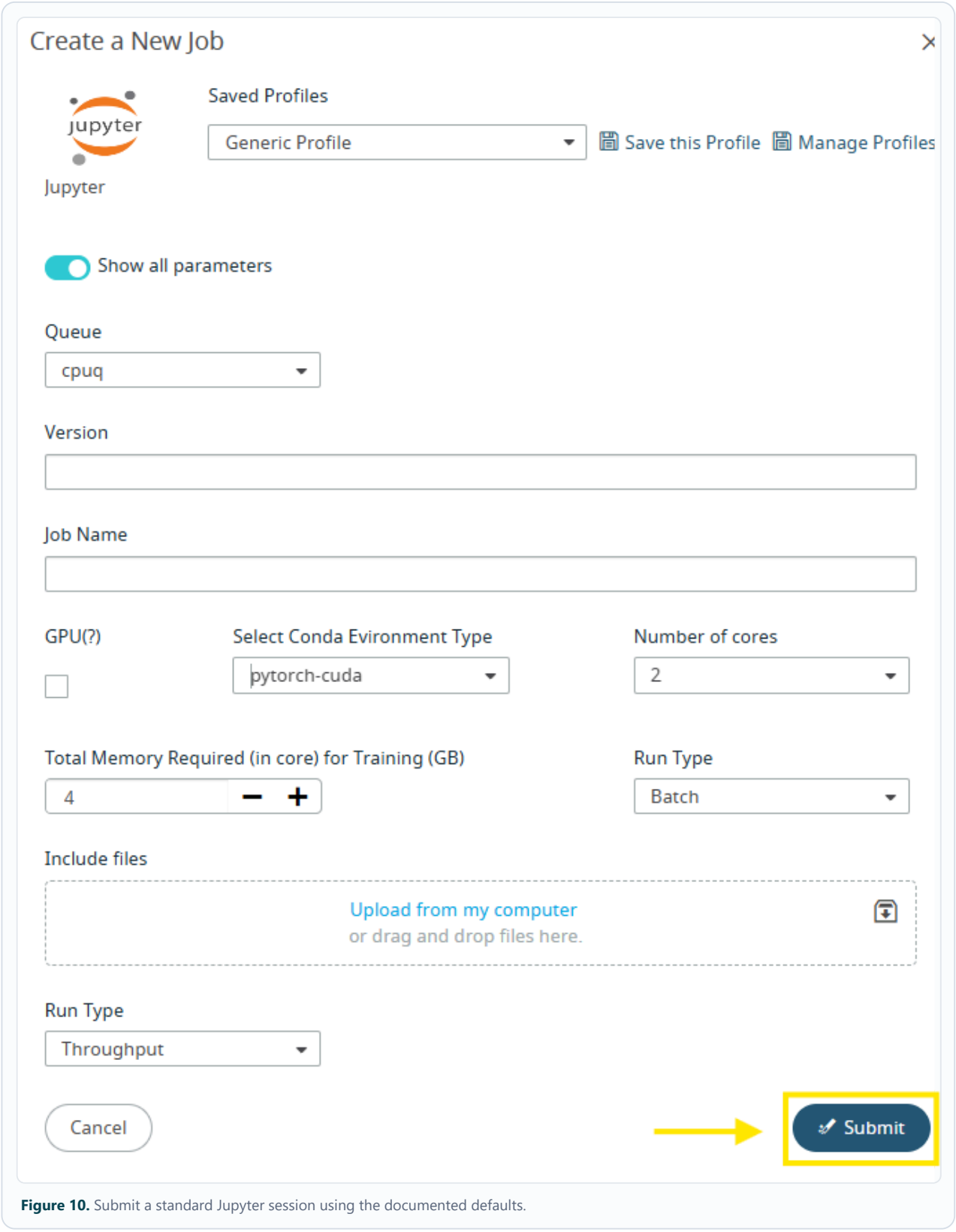


Figure 10. Submit a standard Jupyter session using the documented defaults.

3 Wait for the job to run

Return to **Jobs**, locate the newly created job for the account, open its Job ID, and wait until the status becomes **Running**. Do not open the Jupyter session before this state appears.

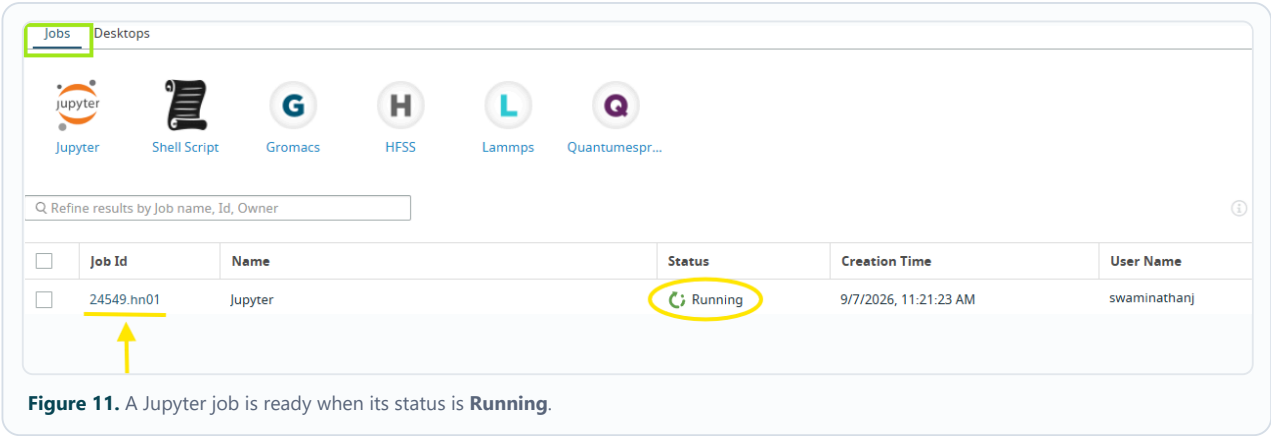


Figure 11. A Jupyter job is ready when its status is **Running**.

4 Open the Jupyter URI

Select **Custom Actions** → **Show Jupyter URI** → **Jupyter Notebook**. The notebook link may be light blue and slightly difficult to notice.

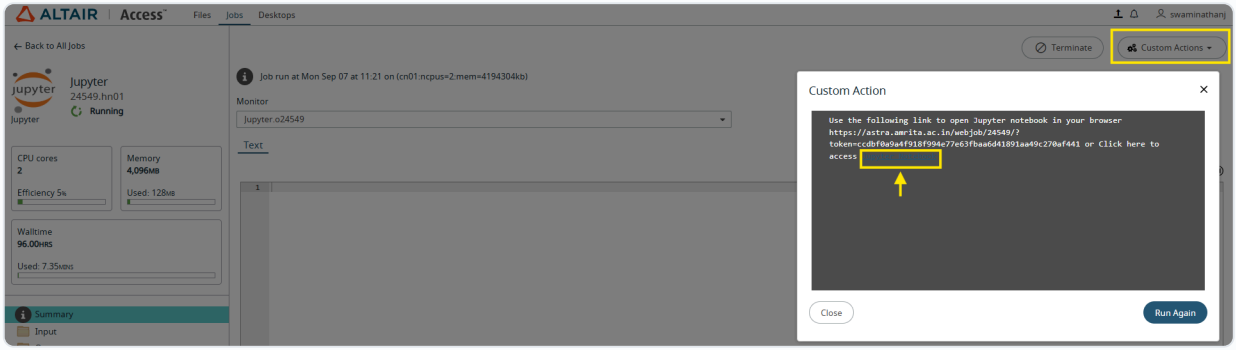


Figure 12. Open the notebook link from the Jupyter URI action.

> **EXPECTED INTERNAL CERTIFICATE WARNING**
>
> If the browser displays **Your connection is not private** for the documented internal Astra session, use **Advanced** → **Proceed** as described in the source instructions.

8. Accessing the Terminal

1 Open a terminal

Inside the Jupyter Notebook interface, select **New** → **Terminal**.

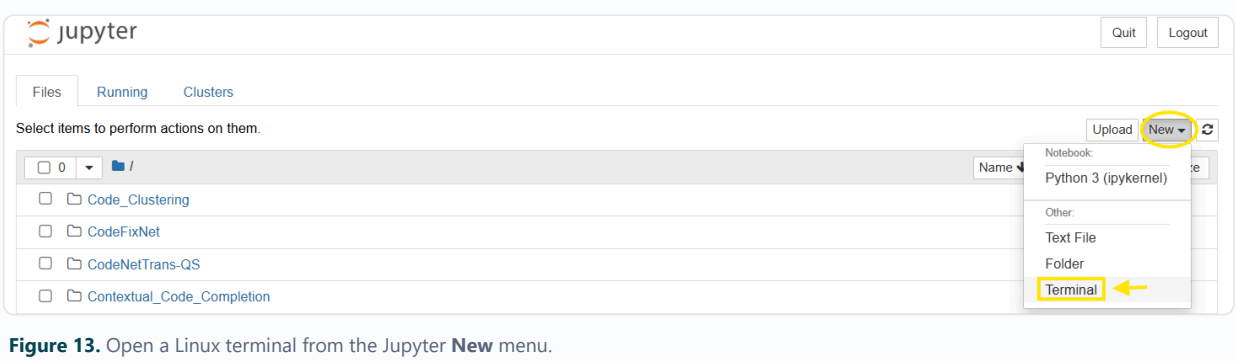


Figure 13. Open a Linux terminal from the Jupyter **New** menu.

2 Navigate to the project

The terminal may open inside a Jupyter-generated working directory. The original workflow uses `cd ..` before entering `Project1`. An absolute path is more reliable:

```bash
cd /home/<username>/Project1
ls
```

Confirm that `requirements.txt` and the expected project folders are present. If the dependency file is stored under the recommended content layout instead, navigate to its exact location, such as `/home/<username>/Project1/content/files`.

9. Creating a Python Virtual Environment

Create an isolated environment named `env` once for the project, then activate it:

```bash
cd /home/<username>/Project1
python3 -m venv env
source env/bin/activate
```

After activation, the prompt normally begins with `(env)`. The interpreter is stored at `/home/<username>/Project1/env/bin/python`.

> **SUBSEQUENT SESSIONS**
> The environment only needs to be created once. Later, return to the project directory and run `source env/bin/activate`.

10. Installing Dependencies

With the virtual environment active, install the non-PyTorch dependencies listed in `requirements.txt`:

```bash
pip install -r requirements.txt
```

Wait for installation to complete successfully. Keep `torch`, `torchvision`, and `torchaudio` out of this file so a generic package resolution does not replace the tested CUDA build.

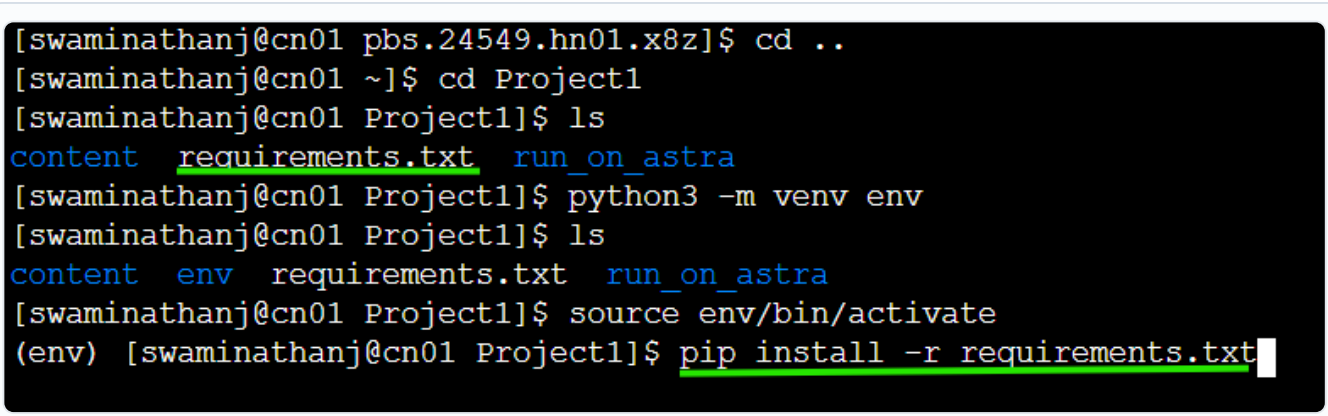


Figure 14. Creating and activating `env`, then installing `requirements.txt`.

11. Installing the CUDA-Compatible PyTorch Build

The documented Astra configuration uses NVIDIA A100 GPUs and has been verified with NVIDIA driver `520.61.05`; `nvidia-smi` reports CUDA compatibility `11.8`. Install the tested PyTorch 2.5.1 CUDA 11.8 wheels separately:

```bash
python -m pip install --upgrade pip setuptools wheel
pip uninstall torch torchvision torchaudio -y
pip install torch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 \
    --index-url https://download.pytorch.org/whl/cu118
```

What the CUDA version indicators mean

INDICATOR	WHAT IT TELLS YOU
`nvidia-smi`	Reports the NVIDIA driver and the highest CUDA compatibility level supported by that driver—not necessarily an installed toolkit version.
`nvcc --version`	Reports the installed CUDA toolkit compiler, if `nvcc` is present in that environment.
`torch.version.cuda`	Reports the CUDA runtime version against which the installed PyTorch package was built.
`torch.cuda.is_available()`	Reports whether this PyTorch process can currently access a compatible CUDA GPU.

> **VERSION INTERPRETATION**
> These values describe different layers and do not all need to be identical. The important outcome is that the driver supports the selected PyTorch runtime and that the scheduled process can access its allocated GPU.

12. Verifying PyTorch and GPU Availability

Run verification from a scheduled GPU job so that the test reflects a GPU compute allocation. A compact command is:

```bash
python -c "import torch; print(torch.__version__); print(torch.version.cuda); print(torch.cuda.is_available())"
```

For a reusable test program, use:

```python
import torch

print("GPU Availability:", torch.cuda.is_available())
print("Torch Version:", torch.__version__)
print("PyTorch CUDA Version:", torch.version.cuda)

if torch.cuda.is_available():
    print("Visible GPU Count:", torch.cuda.device_count())
    print("GPU Name:", torch.cuda.get_device_name(0))
```

A successful GPU allocation should report `GPU Availability: True` and identify the allocated GPU model.

> **GPU ALLOCATION BOUNDARY**
> `torch.cuda.device_count()` reports devices visible to the process. Visibility may not necessarily be identical to the number requested. Do not use all visible GPUs unless the job explicitly requested and was allocated those GPUs.

Multi-GPU execution requires code designed for multi-GPU training or inference. Merely requesting additional devices does not parallelize a single-GPU program.

13. Preparing the Main Python Program

Place the actual experiment or application in the project source directory. For the first test, create:

`/home/<username>/Project1/content/files/test.py`

Use the GPU verification program from Section 12. For a production workload, replace the test logic with the main program while retaining explicit paths for datasets, models, and outputs.

> **KEEP THE ROLES CLEAR**
> The main Python script contains the research workload. The scheduler script is a small wrapper that launches it with the correct virtual-environment interpreter.

14. Creating the Scheduler/Wrapper Script

In `/home/<username>/Project1/run_on_astra`, create `run_test.py`. Replace `<username>` with the Astra account username:

PYTHON

```python
#!/usr/bin/env python3

import subprocess

print("Start running on Astra", flush=True)

which_python = "/home/<username>/Project1/env/bin/python"
main_path = "/home/<username>/Project1/content/files/test.py"

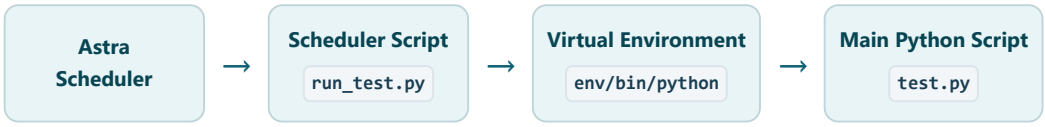
subprocess.run([which_python, main_path], check=True)

print("Done running on Astra", flush=True)
```

`which_python` selects the interpreter containing the installed dependencies; `main_path` identifies the program to run. `check=True` makes the wrapper fail clearly when the main program returns an error.

USE ABSOLUTE PATHS

Scheduled jobs can start in generated working directories. Absolute paths prevent failures caused by an unexpected current directory.



15. Scheduling a GPU Job

1 Open Shell Script submission

Return to the main Astra interface and select **Jobs** → **Shell Script**.

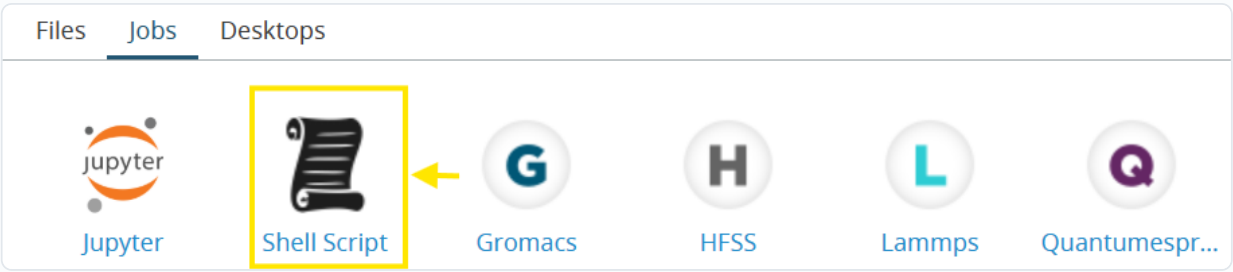


Figure 15. Start a scheduler-script submission from **Jobs** → **Shell Script**.

2 Reveal GPU parameters

Turn on **Show all parameters** to display the GPU and queue settings.

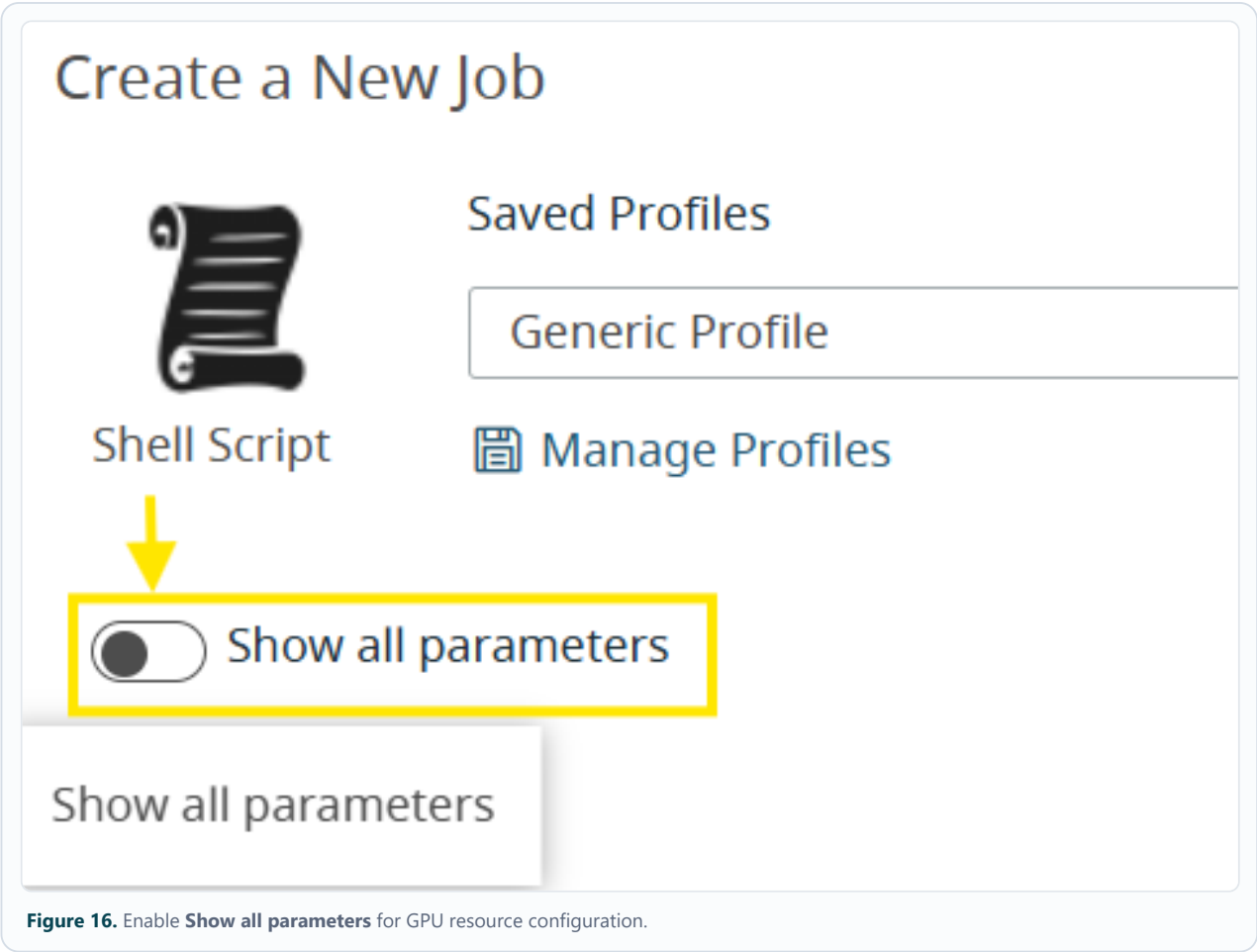


Figure 16. Enable **Show all parameters** for GPU resource configuration.

3 Configure the job

Give the job a descriptive name such as `Project1_GPU_Test`, enable GPU use, select `gpuq`, and request only the necessary number of GPU cards.

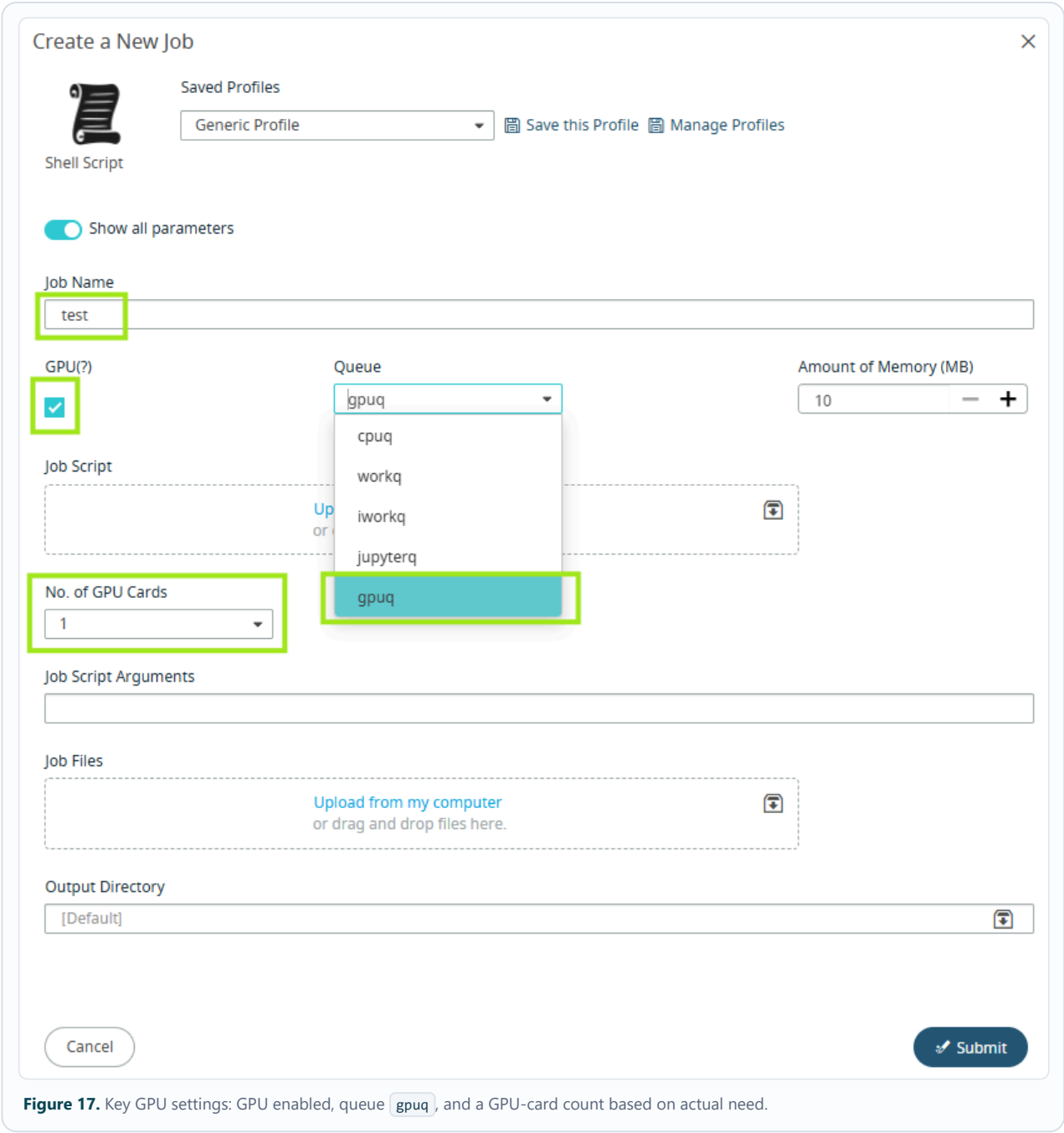


Figure 17. Key GPU settings: GPU enabled, queue `gpuq`, and a GPU-card count based on actual need.

4 Select the wrapper script

In the file browser on the right, navigate to `/home/<username>/Project1/run_on_astra`. Drag `run_test.py` into the **Job Script** field. Submit the wrapper—not `test.py` directly.

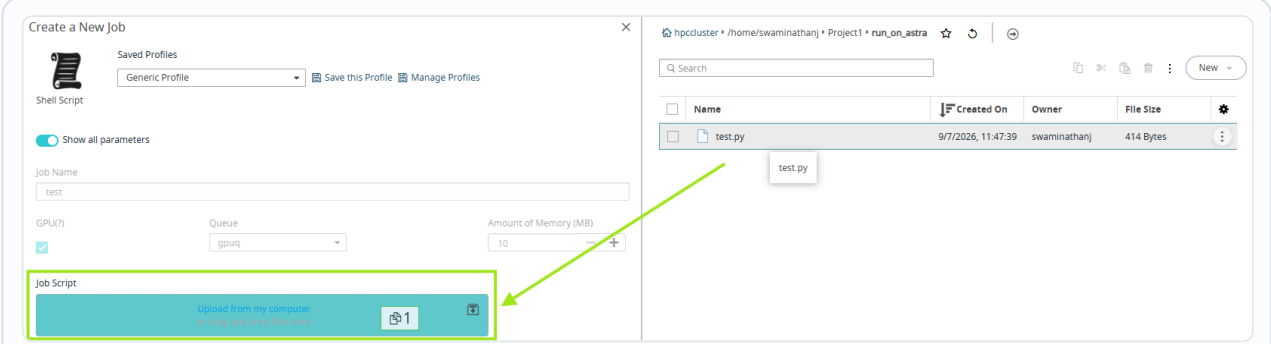


Figure 18. Drag the scheduler/wrapper file from the right-side browser into **Job Script**.

5 Submit

Review the configuration and click **Submit**. Astra creates the job and sends it to the scheduler.

16. Selecting Queue and GPU Resources

FIELD	DOCUMENTED SETTING	GUIDANCE
Job Name	`Project1_GPU_Test` (example)	Use a descriptive name that is easy to identify in the Jobs tab.
GPU option	Enabled	Required for the documented GPU test.
Queue	`gpuq`	Routes the submission to the documented GPU queue.
Memory (MB)	Default retained in the example	Use the documented default for this setup; request appropriately for real workloads.
No. of GPU Cards	1–6	Select only what the program requires and supports. Use one for ordinary single-GPU programs.
Job Script	`run_test.py`	Submit the scheduler wrapper that invokes the virtual-environment Python.

> **!** **SHARED GPU USAGE**
>
> For a normal single-GPU program, request one GPU. Astra provides up to six A100 devices, but requesting more GPUs does not automatically improve performance and may increase queue time.

17. Monitoring a Running Job and Viewing Output

Open the **Jobs** tab to monitor the submission. The scheduler may show:

Queued
Waiting for the requested resources to become available.

Running
The program is currently executing.

Completed
The program finished successfully.

Failed
The job encountered an error; inspect its output and logs.

1 Open the job details

After submission, open the **Jobs** tab and select the corresponding job entry. If its status is **Running**, click the job to open its details.

2 Use the Monitor tab

In the job details page, locate **Monitor**. Its drop-down menu contains files generated or retained for the job, which may include:

- `.o` output files—often named like `test.o24558`—containing standard console output.
- `.e` error files—often named like `test.e24558`—containing execution errors.
- `access.log` for access-related log information.
- The submitted scheduler script, such as `start.py` or `test.py`.
- Other files generated by Astra/PBS, such as the job environment file.

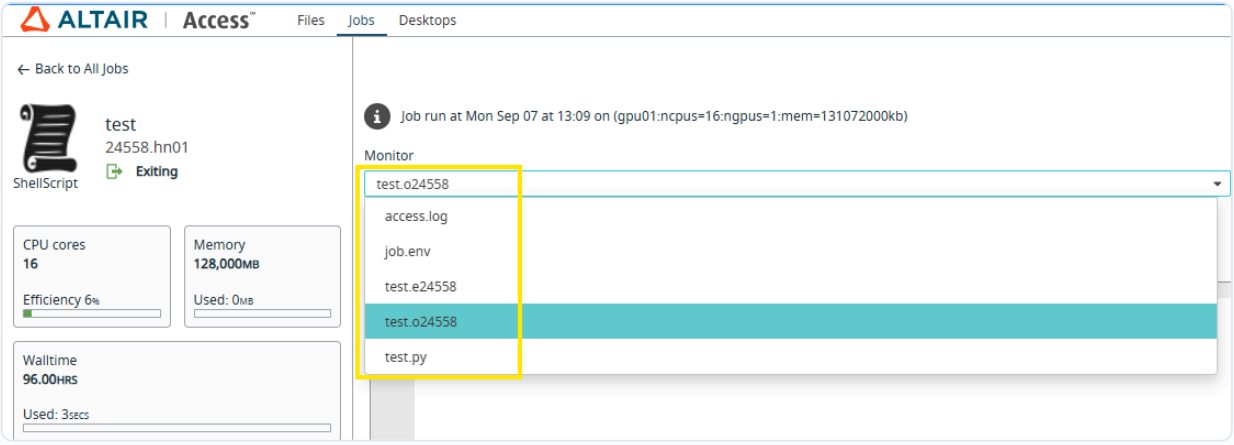


Figure 19. The **Monitor** menu lists the job's generated files. The example shows `test.o24558` and `test.e24558`, where the numeric suffix is the job identifier.

3 View program output

Select the `.o` file. It displays console output from the program, including Python `print()` statements and diagnostic information. The documented GPU test produces output similar to:

```
JOB OUTPUT
start.py: No #! line detected in Job Script, defaulting to Python.
GPU Availability: True
Torch Version: 2.5.1+cu118
Start running on HPC
Done running on HPC
```

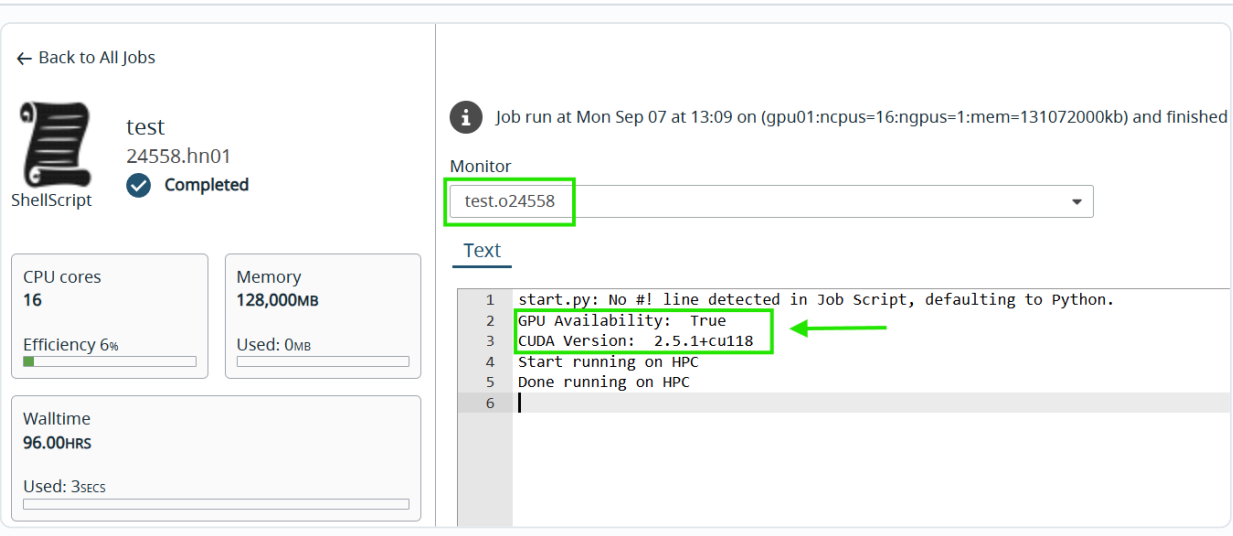


Figure 20. Standard output from the completed GPU test confirms GPU visibility and successful execution.

This result confirms that the scheduler wrapper ran, the virtual-environment interpreter was used, PyTorch detected a GPU, the installed build is compatible with the documented CUDA 11.8 environment, and the main program completed.

ABOUT THE MISSING-SHEBANG MESSAGE

`No #! line detected in Job Script, defaulting to Python` appears when the submitted Python file has no shebang line. Astra recognizes it as Python automatically, so this message is generally informational rather than an error. The wrapper template in Section 14 includes a shebang to make the interpreter intent explicit.

INTERPRET THE VERSION LABEL CAREFULLY

The screenshot labels `2.5.1+cu118` as "CUDA Version," although that format identifies the PyTorch package build. Use `torch.__version__` for that value and `torch.version.cuda` for PyTorch's bundled CUDA runtime.

View errors

If the job fails or produces unexpected results, select its `.e` file first. It may contain:

- Missing Python packages or Python exceptions
- Incorrect file paths or permission errors
- CUDA or GPU-related errors
- Failed subprocess execution

RECOMMENDED REVIEW ORDER

After each job finishes, inspect the `.o` file for program output. If the job failed or expected output is missing, inspect the corresponding `.e` file for error details.

Generated folders and logs

Astra may create job-related directories in the scheduler script's launch location. Timestamped `Jupyter_...` folders under `/PBSHOME/stage/<username>` are also associated with scheduled sessions and logs. This is why program content belongs under `/home/<username>` and wrapper scripts are best isolated in `run_on_astra`.

18. Common Issues and Troubleshooting

ISSUE	WHAT TO CHECK
Astra login or Jupyter link does not open	Try an Incognito/Private Browsing window. For the documented internal certificate warning, use **Advanced** → **Proceed**.
Jupyter session is unavailable	Confirm that the Jupyter job has reached **Running**, then use **Custom Actions** → **Show Jupyter URI**.
`torch.cuda.is_available()` is `False`	Confirm the job uses `gpuq`, GPU resources were requested, the CUDA 11.8 PyTorch build is installed, and the test is running within the scheduled GPU job.
`nvidia-smi` is unavailable	Run GPU diagnostics within an allocated GPU compute job. A login or setup terminal may not expose GPU devices or diagnostics.
Job remains Queued	The scheduler is waiting for the requested resources. Large GPU requests can take longer to satisfy.
Job Failed	Inspect job output/logs. Verify absolute paths, environment activation/installations, dependencies, selected queue, GPU request, and wrapper-script settings.
File or environment cannot be found	Confirm the project is under `/home/<username>` and that wrapper paths point to the correct `env/bin/python` and main script.

19. Best Practices for Shared GPU Usage

- ✓ Use absolute paths in scheduler scripts.
- ✓ Keep wrapper scripts separate from program source files.
- ✓ Request only the GPUs the program actually uses.
- ✓ Do not use all visible GPUs without an explicit allocation.
- ✓ Do not assume more GPUs automatically improve speed.
- ✓ Avoid unnecessary copies of large datasets and checkpoints.
- ✓ Create a dedicated virtual environment per project.
- ✓ Separate datasets, models, checkpoints, and code.
- ✓ Use one GPU for ordinary single-GPU programs.
- ✓ Verify GPU access inside the scheduled GPU job.
- ✓ Organize files before requesting an ICTS transfer.
- ✓ Keep credentials limited to authorized users.

AFTER-READING CHEAT SHEET

20. Quick Reference

Create and activate the environment

BASH

```bash
cd /home/<username>/Project1
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
```

Install tested PyTorch wheels

BASH

```bash
python -m pip install --upgrade pip setuptools wheel
pip uninstall torch torchvision torchaudio -y
pip install torch==2.5.1 torchvision==0.20.1 \
  torchaudio==2.5.1 \
  --index-url https://download.pytorch.org/whl/cu118
```

Verify PyTorch and CUDA

BASH

```bash
python -c "import torch; print(torch.__version__); \
print(torch.version.cuda); \
print(torch.cuda.is_available())"
```

GPU submission settings

- Queue: `gpuq`
- GPU option: enabled
- GPU cards: actual requirement (normally 1)
- Job Script: scheduler/wrapper Python file

Typical scheduler wrapper

PYTHON

```python
import subprocess

which_python = "/home/<username>/Project1/env/bin/python"
main_path = "/home/<username>/Project1/content/files/test.py"
subprocess.run([which_python, main_path], check=True)
```

Remember

`/home/<username>` is the main workspace. `/PBSHOME/stage/<username>` is associated with scheduled-job execution and logs.